Assignment - 3

NLP: CS60075

Autumn Semester 2025, IIT Kharagpur

Implementing BERT from Scratch for Fake News or Toxicity Classification

Point of contact - Sai Keerthana Karnam (saikeerthana00@gmail.com)

Assignment Beginning: 11:00 PM, 16th October 2025
Assignment Deadline: 11:59 PM, 5th November 2025

Plagiarism Alert

- Plagiarism in any form is strictly prohibited.
- Both the SOURCE and the TARGET individuals will suffer a penalty of 100% if a plagiarism detector tool detects 60% or more code similarity.
- You are not supposed to share your assignment with anyone even after the deadline for submission is over.
- We shall use multiple AI detectors like GPT (and 3 more undisclosed), and if at least 3 of them flag your code as AI-generated then we shall deem the submission to be 100% plagiarism.

[**For this assignment, it is strongly recommended that you utilize free GPU Accelerators from Kaggle or Google Colab.** These will be highly beneficial for training your models within a reasonable timeframe.]

Overview

In this assignment, you will implement an encoder-only Transformer (BERT) from scratch to gain a deep understanding of its core components.

Next, you will work with the WildChat-1M-Full dataset, which contains conversations between humans and ChatGPT. Your task is to classify as toxic or non-toxic.

OR

You will work with the Fakeddit dataset, which contains the posts from reddit and the comments on the respective posts. Your task is to classify whether the post is fake or not.

⚠️ **Note: Please request access to the [WildChat-1M-Full](#) dataset immediately. Do not wait until the last moment, as getting the permission to access this dataset may take time. Mention in the report clearly which dataset you are using.**

Tasks

Task 1: Implement BERT class from scratch **[40% of TOTAL]**

Objective: Implement the essential components of a BERT model from scratch using PyTorch, and test it on a sample sentence. The main goal of this task is to understand the data flow and the shapes of the embeddings. I've provided the references to guide your understanding. Please use them for reference only, not as-is.

Requirements:

1. **Implement the following components in PyTorch:** Consider number of heads = 12, embedding dimension = 768, number of layers = 2
    - Multi-Head Self-Attention
    - Feed-Forward Layer
    - Transformer Encoder Layer
    - BertModel (stack of encoders with token embeddings, positional embeddings, pooler, and classifier head)
2. **Tokenizer:** Use `bert-base-uncased` from Hugging Face to tokenize inputs.
3. **Testing with a Sample Sentence:**
    - Consider any sentence of your choice with **more than 10 words**.
    - Tokenize the sentence using the tokenizer.
    - Pass the tokenized input through your implemented BERT model and for the segmentation information (if required) pass 0 for all the tokens.
4. **Report:**
    - Record and include the **shapes of embeddings** at each stage of the model:
        - Token embeddings
        - Positional embeddings
        - Output of Multi-Head Attention
        - Output of Feed-Forward layer
        - Output of Transformer Encoder Layer
        - Output probabilities from the classifier
        - Shape of each parameter.

Task 2: Data Preprocessing **[20% of TOTAL]**

WildChat

Prepare the WildChat-1M-Full dataset for toxicity classification by filtering, cleaning, and organizing the data.

Steps:

1. Filter by Language:
    - Use the *language* column in the dataset and retain only conversations in English.
2. Use the column 'conversation' to access the user prompts and the response from the ChatGPT. Anonymize the chat conversation by giving the input as the conversation chains between User A and User B, where User A = human, User B = chatGPT.
    - Report the input format you are using.
3. **[BONUS - 10% of TOTAL]** Apply data cleaning strategies (example removing duplicates), mention the methods used and the data size after each step.
4. Balance the Dataset:
    - The `toxic` column contains whether the conversation contains toxic or not. Use it as the ground truth
    - Create a balanced dataset by randomly sampling equal numbers (consider at least 2500 each) of toxic and non-toxic conversations.
5. Train-Test Split:
    - Split the filtered dataset into 80% training and 20% testing.
    - Make sure that both sets have balanced classes.
6. Report Statistics:
    - Number of English conversations in the original dataset
    - Number of toxic and non-toxic conversations after filtering.
    - Number of toxic and non-toxic conversations in the balanced dataset you built.
    - Number of conversations in train and test sets

Fakeddit

Prepare the Fakeddit dataset for fake news classification by filtering, cleaning, and organizing the data.

Steps :

1. Navigate to the folder *all samples (also includes non multimodal)*. Load and combine the files - *all_train.tsv, all_test_public.tsv and all_validate.tsv*. Retain only the columns 'clean title' for the post content, '2_way_label' for the classification and 'id' for identifying the post.
2. **[BONUS - 10% of TOTAL] Post–Comment–Reply Structure:** Obtain the 'all_comments.tsv' file from the Fakeddit GitHub repository. Load the following columns:

- `id` → Unique comment ID
- `submission_id` → ID of the post this comment belongs to
- `body` → content of the comment
- `parent_id` → Parent of the comment
    - Prefix `t3_` → Comment directly on the post (top-level comment)
    - Prefix `t1_` → Reply to another comment

Use this information to add comments to the inputs to enrich the text. Mention the format you are using to feed to the model. Note that you will still classify whether the post is fake or not.

3. Balance the Dataset:
    a. Create a balanced dataset by randomly sampling equal numbers (consider at least 2500 each) of fake and non-fake posts.
4. Train-Test Split:
    a. Split the filtered dataset into 80% training and 20% testing.
    b. Make sure that both sets have balanced classes.
5. Report Statistics:
    a. Number of posts in the original dataset
    b. Number of posts which are fake and non-fake in the balanced dataset you built.
    c. Distribution of posts in train and test sets.
    d. For those who did step 2 mention the following apart from above statistics:
        i. Number of posts with at least one comment in the original dataset
        ii. Mean, standard deviation of number of comments for fake vs non-fake posts in the entire dataset.
        iii. Mean, standard deviation of number of comments for fake vs non-fake posts in the balanced dataset you built.

Task 3: Fine tune the model and Report Metrics [40% of TOTAL]

Steps :

1. For this part, there is no need to use the **class you implemented in Task 1.**
2. Use `BertForSequenceClassification` from transformers and load the model *bert-base-uncased* to finetune the model.
3. If you are using only a subset of the input tokens, specify the exact values in your report and clearly describe the method used for truncating the input.
4. Set Hyperparameters and Train:
    a. Define hyperparameters (epochs, batch size, learning rate, optimizer).
    b. Finetune the model using cross-entropy loss on the training set.
5. Evaluate Performance:
    a. Test the model on the test set and compute Accuracy, Precision, Recall, F1-score, confusion matrix.
6. Report Results:

a. Present a metrics table for the test set.
b. Document all hyperparameters used.

References

- https://medium.com/@whyamit101/fine-tuning-bert-a-practical-guide-b5c94efb3d4d
- https://medium.com/data-and-beyond/complete-guide-to-building-bert-model-from-sratch-3e6562228891

Submission Guidelines

- You have to use IPython Notebooks for coding. (Use Google Colab, Kaggle for running your assignments).
- Include small documentation on each cell of the notebook. Further, properly add comments to your code as and when required.
- **Important –** Make sure your IPython Notebook has the outputs from each cell. The outputs must be present in the submitted notebook as well. Absence of these results will lead to straight **deduction of 50% marks.**
- You have to submit the IPython notebook and Assignment report as mentioned above. Name each of them as NLP_Assignment_3_Roll_No.ipynb and NLP_Assignment_3_Roll_No.pdf, respectively.

  [**For eg:** NLP_Assignment_3_21CS30035.ipynb, NLP_Assignment_3_21CS30035.pdf]

- Submit these two files in **.zip format** on **Google Form**. **Note**: Your zip file must be named NLP_Assignment_3_Roll_No.zip and if you submit more than once, only your **latest submission** will be considered.

  [**For eg:** NLP_Assignment_3_21CS30035.zip]